

UNIT-IV

Security in Ordinary Operating System

Covert Channels

A covert channel is a communication channel not normally used in system communications and is therefore not protected by the system's security mechanisms. This makes it a vulnerability that could be exploited to corrupt a system's security policy. The two common types of covert channels:

Covert storage channels – these transfer data by modifying it on a resource, such as a hard disk drive shared by two subjects at different security levels. This can be achieved by a program transferring information to a less secure program by changing the amount or the patterns of free hard disk space, or by changing the characteristics of a file.

Covert timing channels – covert channels in which one process signals information to another by manipulating one of its observable system resources in such a way that it affects the real response time observed by the second process. This usually manipulates a system clock or timing device. Information is then transferred by using timing measurements such as the duration required to perform an operation, the amount of CPU time expended, or the time occurring between two events.

A covert channel is any communication channel that can be exploited by a process to transfer information in a manner that violates the systems security policy. In short, covert channels transfer information using non-standard methods against the system design.

Information and communication dissimulation is not a new topic. However, applications remain numerous, and most recent techniques make such channels more difficult to detect.

The main purpose of covert channels is to protect privacy or to increase security of critical communication. As with most security concepts, however, covert channels do have a dark side. Given that they provide a stealth and secure communication channel, they can undoubtedly be used to establish connections that are theoretically prohibited by the security policy. Then, information leaks become possible, as well as asynchronous command channels between the compromised system and its master.

Covert channels are not everywhere, but they can be everywhere, thus providing answers to several issues raised using encryption: legal restrictions and lack of discretion. In the first case, the main concern is the protection of personal privacy. People want their communications not to become public. The second issue is to have communications remaining undetected. If the content of an encrypted mail should not be readable, the communication itself is not stealth. And this piece of information may be valuable, mainly if one knows that two entities tried to protect the privacy of their communication.

Today, covert channels and their technological side – steganography – represent the new frontier of cyber-crime and cyber-espionage. To defend against these channels, you must understand how they work. Once we know what we're up against, we can take decisive action. My presentation next week at CSX Europe will cover the main aspects of covert channels and steganography. The objective is to explain how they work, how to detect them and which countermeasures a company must take to prevent them using many practical examples.

Creating a covert channel takes some ingenious programming, and often access to the file system at the source end of the communication is essential. Covert channel analysis is one of the few ways to detect a covert channel. System performance degradation can be used to show covert channel use, but as computers have advanced, the degradation is insignificant compared to the amount of data processed. This makes detection even harder.

The primary way of defending against covert channel attacks is to examine the source code running on the source machine, as well as monitor resource use by the system in question.

UNIX security

User Accounts

Every UNIX-like system includes a root account, which is the only account that may directly carry out administrative functions. All the other accounts on the system are *unprivileged*. This means these accounts have no rights beyond access to files marked with appropriate permissions, and the ability to launch network services.

Network Ports: Only the root account may launch network services that use port numbers lower than 1024. Any account may start network services that use higher port numbers.

Each user should have a single account on the system. Network services may also have their own separate accounts, in order to be able to access those files on the system that they require. Utilities enable authorized users to temporarily obtain root privileges when necessary, so that administrators may manage the system with their own user accounts.

Avoid Logging in as root: You do not need to log in with the root account to manage any aspect of your system. Use tools such as `su` or `sudo` when you need to carry out an administrative task that requires root privileges.

For convenience, accounts may be members of one or more *groups*. If a group is assigned access to a resource, then every member of the group automatically has that access.

User Private Groups: On many distributions, each account is automatically made the sole member of a group with the same name as the account. This enables you to easily limit access to files or directories, by associating them with a group that will only ever have one member.

The majority of UNIX-like systems use a *Pluggable Authentication Modules (PAM)* facility to manage access by users. For each login attempt or password change, the relevant service runs the configured *PAM* modules in sequence. Some modules support authentication sources, such as locally stored files or *LDAP* directory services. Administrators may enable other modules that carry out setup tasks during the login process, or check login requests against criteria, such as a list of time periods when access is permitted.

File Permissions

Every file and directory on a UNIX-style system is marked with three sets of file permissions that determine how it may be accessed, and by whom:

- The permissions for the *owner*, the specific account that is responsible for the file.
- The permissions for the *group* that may use the file.
- The permissions that apply to all *other* accounts

Each set may have none or more of the following permissions on the item:

- *read*
- *write*
- *execute*

A user may only run a program file if they belong to a set that has the *execute* permission. For directories, the *execute* permission indicates that users in the relevant set may see the files within it, although they may not actually read, write, or execute any file unless the permissions of that file permit it. Executable files with the *set UID* property automatically run with the privileges of the file owner, rather than the account that activates them. Avoid setting the execute permission or set UID on any file or directory unless you specifically require it.

root Ignores File Permissions: The root account has full access to every file on the system, regardless of the permissions attached to that file.

The majority of files on a UNIX-like system are owned by the root account and have permissions that restrict or block access from all other accounts. Avoid modifying the permissions on system files and directories.

Historically, user home directories on UNIX-like systems were publicly readable, to facilitate sharing between academic colleagues. Unfortunately, some popular operating systems still make user home directories publicly readable by default.

Access Control Lists: Many, but not all, modern UNIX-like systems include support for a more flexible set of permissions known as *Access Control Lists* (ACLs). Unfortunately, some common applications are not fully compatible with ACL permissions.

Data Verification

To create a checksum for a file, or to test a file against a checksum, use the *sha1sum* utility. SHA1 supersedes the older MD5 method, and you should always use SHA1. For more information about about *sha1sum*, refer to the manual:

```
man sha1sum
```

Open Source UNIX-like systems also supply the GNU Privacy Guard (GnuPG) system for encrypting and digitally signing files, such as emails. Many documents refer to GnuPG as *gpg*, which is the name of the main GnuPG command.

GnuPG Follows the OpenPGP Standard: The files that you sign or encrypt with GnuPG are compatible with other applications that follow the *OpenPGP* standard.

The Evolution email application automatically supports both signing and encrypting emails with GnuPG. Evolution is the default email application for several of the main Linux distributions, including Fedora, Novell Linux Desktop, Red Hat Enterprise Linux, and Ubuntu.

To use other GnuPG features in the GNOME desktop environment, install Seahorse through the standard software management tool for your distribution. For more information on Seahorse, refer to the project Web site:

<http://www.gnome.org/projects/seahorse/>

To integrate GnuPG with Mozilla Thunderbird, add the Enigmail extension to Thunderbird. Refer to the Enigmail Web page for installation instructions and other details:

<http://enigmail.mozdev.org/>

For more information on GnuPG itself, refer to the project Web site:

<http://www.gnupg.org/>

Encrypted Storage

Create one or more *encrypted volumes* for your sensitive files. Each volume is a single file which may enclose other files and directories of your choice. To access the contents of the volume, you must provide the correct decryption password to a utility, which then makes the volume available as if it were a directory or drive. The contents of an encrypted volume cannot be read when the volume is not mounted. You may store or copy encrypted volume files as you wish without affecting their security.

The cross-platform Truecrypt utility enables you to create and access your encrypted volumes with all popular operating systems:

<http://www.truecrypt.org/>

In extreme cases, you may decide to encrypt an entire disk partition that holds or caches data, so that none of the contents may be read by unauthorized persons. On Linux you may use either LUKS, CryptoFS, or EncFS to encrypt disks. Unfortunately, many UNIX-like systems do not yet integrate support for disk encryption facilities into their installation and management software, which makes configuration and maintenance more difficult. Disk encryption also reduces performance, and this may not be acceptable for systems that run demanding applications.

Secure Remote Access with OpenSSH

Every common UNIX-like system today includes a version of *OpenSSH*, an implementation of the *SSH* standard for secure remote access. An SSH service uses strong encryption by default, and provides the following facilities:

- Remote command-line access
- Remote command execution
- Remote access to graphical software
- File transfers

In addition, the forwarding features of SSH allow you to *tunnel* connections to other services through SSH. A tunneled service benefits from the same security and data compression features as the built-in facilities of SSH. This enables you to protect almost all communications between any UNIX-like systems, even when the traffic passes over open wireless networks or the public Internet.

SSH software not only encrypts the connection between systems, but also uses a system of keys to provide *mutual authentication* between each party. Each SSH client utility automatically checks the identity of any remote system that it connects to, by verifying the key. Similarly, users may identify themselves to systems with a key, rather than typing potentially crackable passwords.

Use SSH by Default: SSH potentially offers the most secure method of remote access available today. The standard Open-Source desktop environments now also support SSH as a standard method for working with remote files. Only enable access to your systems through other services if you need to do so to meet a specific requirement.

Most Linux distributions include the OpenSSH client by default. The OpenSSH service is usually offered as an option, although some distributions also provide it by default. macOS includes both the OpenSSH service, and the client utilities. To access SSH services from Microsoft Windows systems, install PuTTY. You may download PuTTY from the main project Web site:

<http://www.chiark.greenend.org.uk/%7Esgtatham/putty/>

FileZilla, WinSCP, and other file transfer utilities for Microsoft Windows also support SSH.

The OpenBSD project maintains the OpenSSH software and Web site:

<http://www.openssh.com/>

Software Management

The majority of Linux distributions incorporate software management facilities based on *package* files and sets of specially prepared Web sites, known as *repositories* or *channels*. Package management utilities construct, or update working copies of software from these packages and execute any other setup tasks that packages require. Repositories enable the management tools to automatically provide the correct version of each software product, by downloading the required packages directly from the relevant repository. Most systems also use checksums and digital signature tests to ensure that packages are authentic and correct.

In addition, package management tools can identify outdated versions of software by checking the software installed on a system against the packages in the repositories. This means that you can ensure that all of the supported software on your system does not suffer from a known security vulnerability, simply by running the update routine.

Most desktop systems now automatically alert you when new versions of the installed packages are released to the repositories and provide options to update your system. Graphical interfaces to their software management tools also enable you to browse and select new software from the available packages.

Use packages from repositories whenever possible, to guarantee the provenance of the software on your system, and to ensure that it remains current. If you use software from elsewhere then you will need to verify, install, and update those products yourself. In these cases, download the software directly from the Web site of the manufacturer. Package management tools cannot inventory, check, or maintain any software that was compiled from source code, so you must be particularly careful when you use manually compiled products.

For historical reasons, the main Linux distributions use different package management products. Fedora, Red Hat Enterprise Linux, and related distributions, use the RPM package format, and their software management facility is known as YUM. Debian, Ubuntu, and their derivatives, use the APT management system and the DEB package format. These systems and package formats are largely equivalent.

Host Integrity Testing

To verify that a running system has not been compromised or tampered with, use an integrity testing facility. All host integrity testing software verifies a complete copy of a system by testing each file against a previously made checksum. Solaris and FreeBSD distributions both now include integrity testing utilities for this purpose. You may also use a cross-platform integrity monitoring system, such as Samhain or Osiris. Both Osiris and Samhain support centralized system auditing for multiple systems.

Since system configurations vary, administrators must configure the integrity tester to exclude the particular directories and files that are expected to change on a system, before creating an initial checksum database for that system. Integrity testing can then compare the checksums of each file against the database, and report on any disparity.

System Recovery

You may easily restore program files for all of the software that is included with your distribution with the software management tools. In order to fully recover a system from accident, or deliberate compromise, you must also have access to copies of the data, configuration, and log files. These files require some form of separate backup mechanism.

All effective backup systems provide the ability to restore versions of your files from several earlier points in time. You may discover that the current system is damaged or compromised at any time and need to revert to previous versions of key files, so keeping only one additional copy of a key file should not be considered an adequate backup.

Duplicates Are Not Archives: File synchronization software and RAID storage make duplicate copies of the current files and may act as a safeguard against data loss from hardware failures. Unlike backup systems, these measures do not provide access to previous versions of files.

Distributions provide a wide range of backup tools and leave it to the administrator to configure a suitable backup arrangement for their systems.

Resource Allocation Controls

You may configure several mechanisms to limit the resources that an application or user account may consume. On systems with multiple users, enforce resource limits to ensure that no user may accidentally or deliberately cause facilities to fail by using all the available resources. Since the correct resource allocations vary widely, administrators must configure appropriate limits for the system.

To set resource limits for particular services, edit the *systemd* configuration file for the service. If you need to limit individual processes, add a *ulimit* setting to the shell script that launches them. For more information about *ulimit*, refer to the manual for the bash shell:

```
info bash
```

The *PAM* login system includes a module to enforce certain resource limits for entire user sessions. The restrictions that this imposes may be circumvented, but they do provide some defence against accidental problems.

You must specifically enable storage quotas on each disk partition, if you require them. Quotas prevent users from overloading the storage and backup facilities, but quota management is often an administrative matter rather than a direct security concern. Configuring storage quotas is beyond the scope of this document.

Monitoring and Audit Facilities

On Linux systems, the *syslog* and *klogd* services record activity as it is reported by different parts of the system. The Linux kernel reports to *klogd*, whilst the other services and facilities on the system send log messages to a *syslog* service. Distributions provide several tools for reading and analysing the system log files.

Several facilities on any UNIX-like system may also email reports and notifications directly to the root account, via the *SMTP* service. Edit the *aliases* file to redirect messages for root to another email address, and you will receive these emails at the specified address.

Automatic Log Summaries: Many distributions automatically send daily reports to the email address for root that summarize the activity logged by syslog and klogd.

To provide a central logging facility for your network, first select one of your systems as the log host. Configure the syslog services on your other systems to forward the information that they receive to the syslog service on the log host. You may then run analyzers on the log host to monitor events across the network.

For detailed real-time monitoring of the systems on your network, install SNMP agents that report to a management service such as Nagios or OpenNMS. SNMP is beyond the scope of this document.

Monitoring Network Appliances: Many network appliances, such as routers, support the syslog and SNMP standards. This enables you to monitor both UNIX systems and other network devices with the same log hosts and SNMP services.

Refer to the man page for basic information about syslogd:

```
man syslogd
```

Similarly, for more on klogd, refer to the man page:

```
man klogd
```

Both syslog and SNMP rely on software dispatching messages to a central service. If you configure *process accounting* on a system, it maintains records of all the processes that are run on that system. Linux includes some support for process accounting, and distributions supply packages for GNU Accounting Utilities. Refer to the Web page for more information on the GNU Accounting Utilities:

<http://www.gnu.org/software/acct/>

Fedora and Red Hat Enterprise Linux systems also offer the LAuS (Linux Auditing System) framework. For more information on this, refer to the man pages for auditd:

```
man auditd
```

The System Firewall

The *netfilter* framework included in the Linux kernel restricts incoming and outgoing network connections according to a set of rules that have been defined by the administrator. Several Linux distributions configure firewall rules by default and offer utilities for managing simple firewall configurations. You may also manage the firewall rules on any Linux system with the standard iptables and ip6tables command-line utilities, or with third-party utilities such as Firestarter. If you decide to use iptables, remember that it only configures restrictions for IP version 4 connections, and that you will need to use ip6tables to setup rules for IP version 6 as well.

Fedora, Red Hat, and SUSE automatically enable the firewall and supply their own graphical configuration utilities. You must manually configure and enable the firewall on Debian and Ubuntu systems. Current releases of Ubuntu include a command-line utility called *ufw* for firewall configuration.

Those Linux distributions that enable a firewall by default use a netfilter configuration that blocks connections from other systems. Any attempt by a remote system to access a service on a blocked port simply fails. This means that no other system may connect to an installed service, unless you specifically choose to unblock the relevant port.

Use Only One Means of Managing Your Firewall: Every firewall utility modifies the current firewall rules on the system. To ensure that your firewall operates correctly, select one method of managing the configuration, and avoid editing the firewall rules by other means.

Application Isolation

The most common UNIX-like operating systems provide several methods of limiting the ability of a program to affect either other running programs, or the host system itself.

- *Mandatory Access Control (MAC)* supplements the normal UNIX security facilities of a system by enforcing absolute limits that cannot be circumvented by any program or account.
- *Virtualization* enables you to assign a limited set of hardware resources to a virtual machine, which may be monitored and backed up by separate processes on the host system.
- *Linux Container* facilities, such as Docker, run processes within a generated filesystem and separate them from the normal processes of the host system.
- The *chroot* utility runs programs within a specified working directory and prevents them from accessing any other directory on that system.

The administrator may setup guest operating systems in virtual environments for specific tasks and restrict these guests far more than would be possible for a multi-purpose system. Each specialized system may include far less software, and this also simplifies every administrative task, including MAC configuration. Neither MAC nor virtualization prevent individual applications or services from being compromised, misconfigured or malfunctioning, but may prevent a problem from escalating.

At the simplest level, the *SELinux* framework can provide MAC facilities, to enforce a policy that defines the access permitted to programs or accounts on the system. SELinux was actually created by the NSA to meet the needs of government agencies handling classified data and enables administrators to develop extremely detailed and precise security configurations that encompass the entire operating system. Many developers and administrators consider SELinux too high a maintenance burden to implement fully.

Fedora and Red Hat Enterprise Linux systems automatically include a limited SELinux policy that restricts many standard network services, without affecting users or other programs. These distributions also provide some simple management tools for customizing the default policy and troubleshooting SELinux issues, but no tools to assist with developing new policies. Debian provides SELinux, but support is limited.

Ubuntu and SUSE do not enable SELinux by default. Instead, they provide the *AppArmor* facility. AppArmor configuration is much simpler than SELinux, but it offers more limited capabilities.

Several Open-Source solutions exist to run complete operating systems within a virtual environment. By far the most popular are *Xen* and *KVM*. Xen enables you to configure a system to act as a host for multiple virtual environments, all of which are controlled by a single hypervisor. Current Linux distributions on machines that include CPUs with virtualization support may run the simpler and more flexible KVM. The current KVM offers significantly higher performance than the QEMU machine emulator that it is based upon. The original QEMU software operates too slowly for production applications, although it remains useful for testing and development work.

Modern Linux systems include support for containers and provide tools that enable you to easily use this facility. Docker relies on a background service that manages the containers on the host system, and this can support a large number of containers on a single host. The *systemd-nspawn* utility that is supplied with *systemd* runs individual containers without requiring an extra service.

Containers Do Not Isolate Processes: By default, any process within a container still has access to the facilities of the host system, such as networking, even though it does not have access to most of the filesystem. Any process that runs as root in a container can alter the host system.

The older *chroot* facility is universally available but was originally designed for development tasks rather than security and may be circumvented. Developers use this facility for building and testing software in a clean environment. Historically, administrators also used *chroot* to run potentially unsafe network services such as FTP servers within specially designed environments. Several tools exist to simplify constructing and maintaining *chroot* environments.

Applications May Escape chroot: Any application that is able to run arbitrary commands can execute code to gain access to the main system. To ensure the security of the *chroot* environment, avoid including shells, compilers, or script interpreters within the *chroot* directory. Any application that runs with root privileges may also escape the restrictions of *chroot*.